Optimized FPGA Acceleration of YOLOv8 Detection Head for Resource-Constrained UAVs

Nguyen Vo, Dang Tran, and Binh Vo

Ho Chi Minh City University of Technology (HCMUT),
Vietnam National University Ho Chi Minh City (VNU-HCM)
{nguyen.vohoang, dang.tran686868, binh}@hcmut.edu.vn

Abstract. UAV-deployed object detectors must satisfy strict Size, Weight, and Power (SWaP) constraints, yet anchor-free models like YOLOv8 introduce severe post-processing bottlenecks, specifically within Distribution Focal Loss (DFL) coordinate decoding and Non-Maximum Suppression (NMS) spatial filtering. Standard Field Programmable Gate Array (FPGA) implementations routinely offload these operations to a host CPU, resulting in memory-transfer penalties that degrade real-time inference. To our knowledge, no prior hardware system implements both post-processing stages entirely within programmable logic. We therefore propose a resource-efficient, fully hardware-integrated YOLOv8 Detection Head accelerator. Specifically, we process all six convolutional layers through a single time-shared MAC engine, decode coordinates via a zero-DSP Look-Up Table (LUT) unit, and filter overlapping bounding boxes by replacing iterative dividers with a parallel algebraic cross-multiplication pipeline. Experimental results on the Xilinx Zynq-7020 platform confirm that the proposed accelerator eliminates the CPU post-processing bottleneck, sustaining a consistent 32.1 FPS at 67.0% mAP_{50} on the KIIT-MiTA dataset within a 0.302 W power envelope using only 10 DSP blocks.

Keywords: FPGA, YOLOv8, hardware architecture, DFL, division-free NMS, UAV, INT8

1 Introduction

Real-time object detection algorithms deployed on Unmanned Aerial Vehicles (UAVs) must operate within strict Size, Weight, and Power (SWaP) constraints [1–3]. Unlike legacy anchor-based networks (e.g., YOLOv5, YOLOv7) that exhibit limited adaptability to multi-scale drone perspectives, modern anchor-free models such as YOLOv8 [4] currently achieve state-of-the-art precision. While subsequent software iterations (e.g., YOLOv10, YOLOv11) propose architectural adaptations, YOLOv8 represents the first widely-adopted anchor-free YOLO design requiring hardware DFL decoding, making it the natural architectural target for this work. This anchor-free design, however, introduces significant algorithmic complexity for fixed-point hardware circuitry. Extracting

bounding-box coordinates requires non-linear floating-point exponentials and normalization division for Distribution Focal Loss (DFL) vector decoding [5], while the mandatory Non-Maximum Suppression (NMS) algorithm conventionally relies on iterative hardware dividers to evaluate the Intersection over Union (IoU) of overlapping bounding-box candidates.

Existing Field Programmable Gate Array (FPGA) accelerators [6, 7] typically prioritize convolutional backbone acceleration, frequently offloading the complex DFL and NMS operations to an embedded Central Processing Unit (CPU) host. Even recent hardware designs explicitly targeting the YOLOv8 Detection Head restrict optimization to architectural modifications (e.g., grouped channel convolutions) [8], leaving the CPU-memory transfer bottleneck unresolved. The resulting latency penalty is substantial. For instance, Danilowicz et al. [9] empirically demonstrated that a generic YOLOv8 FPGA backbone sustaining 195 FPS is bottlenecked to 24 FPS purely due to host-delegated anchor-free NMS synchronization.

Simply offloading these modules to hardware in isolation, however, is insufficient. Integrating these standalone post-processing modules natively into Programmable Logic (PL) presents severe resource challenges. Bespoke NMS hardware pipelines historically rely on extensive hardware dividers [10] or large parallel LUT arrays [11], which rapidly scale DSP and logic capacities beyond the strict limits of Edge UAV deployments. Furthermore, in anchor-free architectures, DFL decoding must complete before NMS can evaluate candidate boxes—a strict sequential dependency that prevents isolated NMS cores from resolving the full post-processing bottleneck without integrated DFL acceleration.

To address these limitations, this paper introduces a resource-efficient, fully hardware-integrated YOLOv8 Detection Head architecture that eliminates the CPU data-transfer bottleneck. The core scientific contributions are:

1. **Zero-DSP LUT-based Distribution Focal Loss (DFL) Decoder:** We propose a 5-stage integer pipeline replacing floating-point Softmax with a 256-entry BRAM Look-Up Table (LUT). This eliminates exponential logic, mapping the full DFL pipeline to precisely 539 LUTs and zero DSP slices.
2. **Division-Free IoU Non-Maximum Suppression (NMS):** We reformulate the IoU calculation to use algebraic cross-multiplication, discarding hardware dividers in favor of a 4-DSP parallel integer multiplier architecture. This reduces the hardware footprint of NMS spatial filtering, enabling autonomous execution within the 220-DSP limit of the Zynq-7020.
3. **Resource-Multiplexed Convolutional Neural Network (CNN) Datapath:** We introduce a time-shared parallel MAC engine that processes all six detection-head layers sequentially using 6 DSPs. Per-channel scale factors are fused directly into the hardware re-quantization path [12], avoiding floating-point corrections and entirely bypassing the CPU during inference bounding-box extraction.

2 System Design

Mapping the YOLOv8 detection head to a fixed-point FPGA entails routing a 40×40 feature map through two parallel paths: a bounding-box regression cv2 branch relying on DFL (4×16 coordinate bins), and a classification cv3 branch, comprising six convolutional layers sequentially. Spatial-parallel instantiation of all six networks often exceeds the logic footprint of edge-class FPGAs. Fig. 1 illustrates our resource-efficient architecture, where the complete head executes autonomously within the PL, fully relieving the ARM Processing System (PS) of post-processing duties.

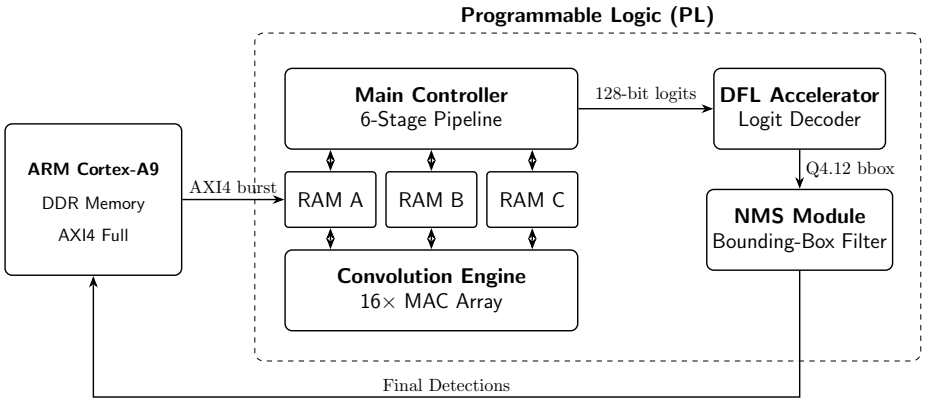

Fig. 1. System Architecture Overview. The PL executes the complete detection head autonomously; the ARM PS is limited to loading input features and receiving final detections via AXI4 burst, eliminating per-frame post-processing overhead.

2.1 Resource-Multiplexed CNN Engine

To minimize the spatial footprint typical of unrolled hardware mappings, our architecture avoids redundant CNN layer instantiations. Instead, we implement a single reconfigurable `conv_stage` supporting variable kernel sizes and channel counts managed by a global Finite State Machine (FSM). The FSM orchestrates execution sequentially (cv2_0 → cv2_1 → cv2_2 → cv3_0 → cv3_1 → cv3_2). Consecutive layer data hazards are accommodated via a 3-BRAM architecture: RAM A and B alternate as ping-pong input/output buffers between consecutive layers, while RAM C independently preserves the cv2 branch output during cv3 branch execution, preventing cross-branch data corruption. This time-multiplexed approach significantly reduces the required logic area compared to fully unrolled designs.

As shown in Fig. 2, the core datapath utilizes an Input Channel Stationary dataflow, routing a BRAM line-buffer into an array of 16 parallel Multiply-Accumulate (MAC) units. By mapping narrow fixed-point multiplications to LUT-based logic rather than hard DSP primitives, the engine restricts DSP utilization to 6 blocks, enabling a 512-bit parallel datapath mapped primarily to

standard logic elements. Quantization scaling is implemented directly in hardware as:

$$y = \text{clamp}\left(\left(\frac{(\text{acc} - z_{in}) \times M_{int}}{2^S}\right) + z_{out},\ -128,\ 127\right) \tag{1}$$

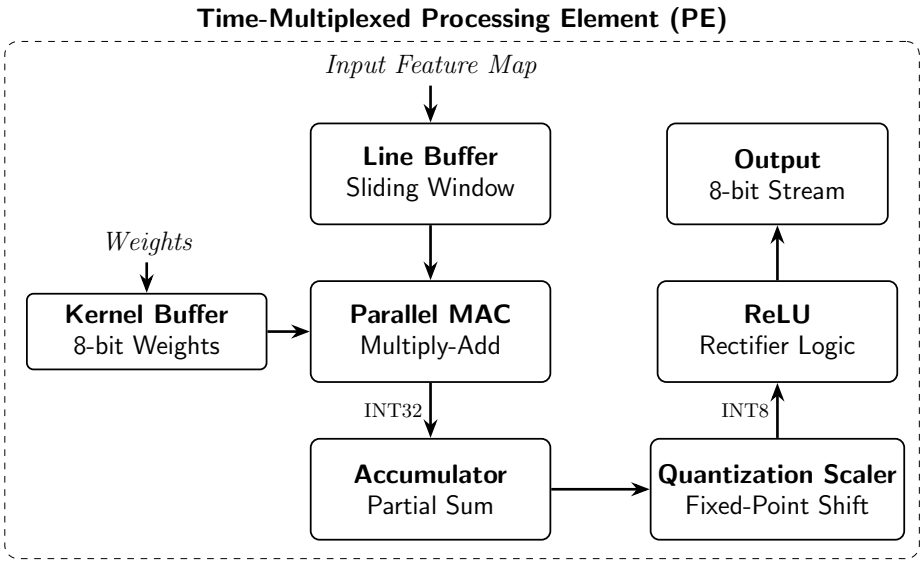

Fig. 2. Processing Element (PE) Microarchitecture. The PE array generates a 512-bit output per cycle via 16 parallel MAC units, with re-quantization performed entirely in combinational logic without DSP primitives.

2.2 Zero-DSP Hardware DFL Accelerator

DFL converts 16 INT8 logits into a single fixed-point bounding-box coordinate. Directly mapping DFL to fixed-point hardware is non-trivial: the Softmax operation requires floating-point e^x computation, which typically necessitates costly Taylor-series approximations consuming multiple DSP stages. We resolve this exponential bottleneck by replacing DSP-based Softmax computation with a single-cycle memory lookup sequence, and restricting the subsequent normalization division entirely to standard LUT logic (Fig. 3).

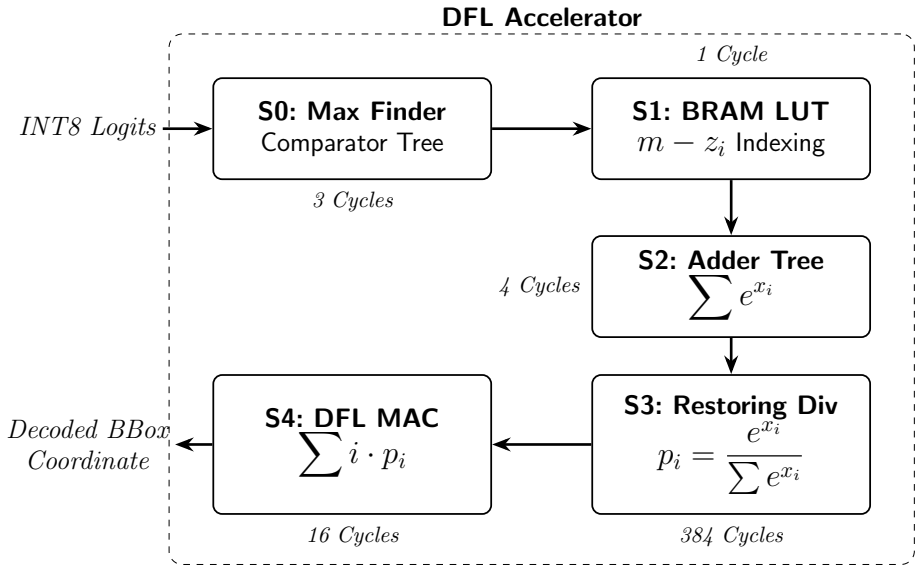

Fig. 3. Five-stage DFL accelerator pipeline. S3 uses a restoring divider implemented in LUT logic, consuming zero DSP slices.

The 5-stage pipelined accelerator isolates the local maximum logit m via a cascading comparator tree (S0) to prevent mathematical overflow ($e^{z_i - m}$). Stage 1 (S1) performs a single-cycle memory fetch against a 256-entry BRAM initialized with Q8.8 fixed-point exponential constants (8 integer bits, 8 fractional bits). This 256-entry depth covers the full value range of an 8-bit quantized logit ($z_{in} \in [0, 255]$), while the Q8.8 fractional format ensures sufficient bounding-box coordinate precision without exceeding a standard 18-bit BRAM width. Stages 2 and 3 accumulate the exponential values and normalize them via a restoring divider, confining operations to standard logic elements. This structure maps the bounding-box coordinate directly to a Q4.12 fractional integer (4 integer bits, 12 fractional bits) without consuming any DSP slices.

2.3 Division-Free IoU NMS

A primary computational bottleneck within standard NMS hardware pipelines is the Intersection over Union (IoU) calculation, which traditionally requires high-latency sequential hardware dividers. Our architecture eliminates this requirement by evaluating a division-free logical formulation:

$$\text{suppress}(B_j) \Leftarrow \text{inter} \times D_{\text{IoU}} > \text{union} \times N_{\text{IoU}} \ \wedge \ \text{conf}(B_i) \geq \text{conf}(B_j) \quad (2)$$

By cross-multiplying both sides of the IoU threshold inequality, division is eliminated entirely, replacing it with two integer multiplications.

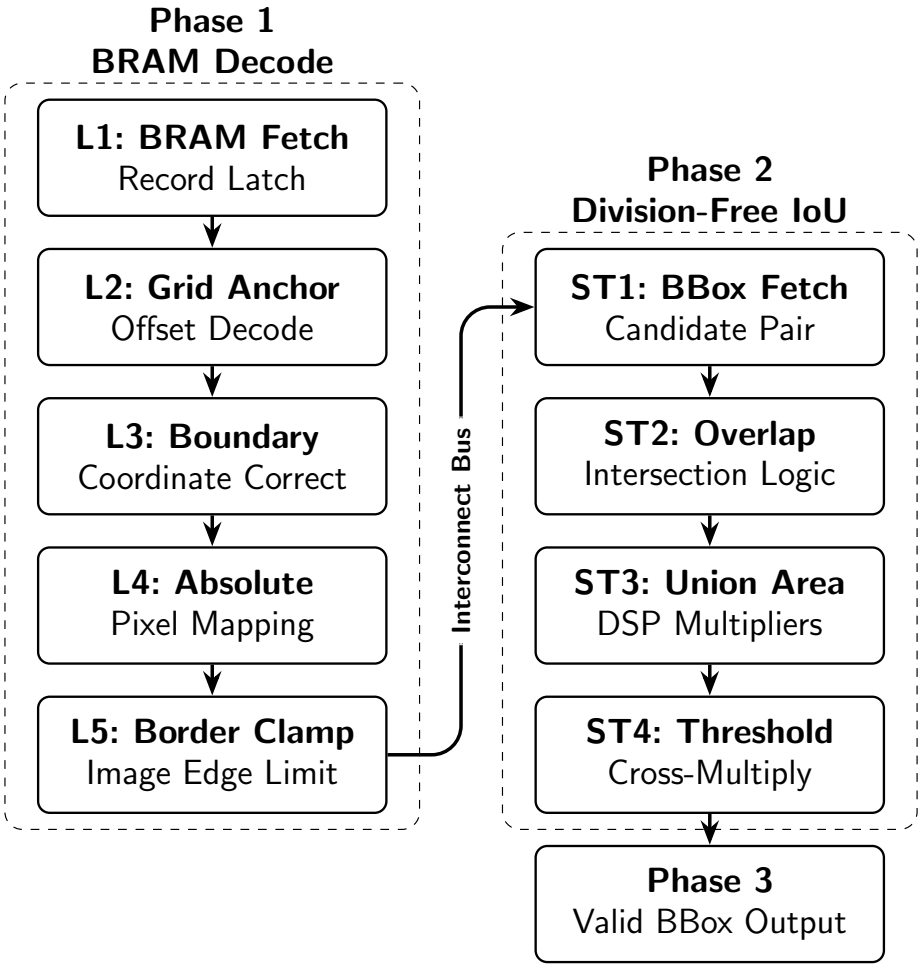

Fig. 4. The Division-free NMS pipeline operates without restoring dividers. Phase 2 (ST1-ST4) evaluates the division-free inequality $I \cdot D > U \cdot N$ by distributing calculations across a 4-DSP array, eliminating all hardware division.

As shown in Fig. 4, the pipeline operates across three architectural phases. Phase 1 unpacks records from BRAM. Phase 2 computes intersecting geometries using exactly three parallel DSP multipliers (Area calculation), while ST4 sequentially executes the $I \cdot D > U \cdot N$ logic upon a fourth DSP. Consequently, the pipeline resolves all threshold comparisons using only 4 DSP slice multipliers, significantly reducing the DSP footprint compared to conventional hardware NMS architectures. Furthermore, overlapping detections are coordinate-averaged rather than suppressed, improving mean IoU by approximately 11% on the test set.

2.4 Quantization Pipeline

All internal hardware paths operate entirely in INT8. A three-phase pipeline integrates Knowledge Distillation (KD) and Per-Channel Quantization Fine-Tuning (PCQ-FT) to bridge the FP32-to-INT8 accuracy gap while satisfying logic constraints.

Phase 1 — Activation Substitution. The standard YOLOv8 SiLU activation ($x \cdot \sigma(x)$) requires a sigmoid lookup and a multiplier at inference time, making it incompatible with a purely combinational INT8 datapath. We substitute SiLU with ReLU ($\max(0, x)$), which maps to a single comparator in RTL. To recover the accuracy lost from this substitution, we apply Knowledge Distillation (KD) from a full-precision SiLU teacher model ($T = 20$, 200 epochs, SGD, $lr = 10^{-2}$, 320×320 input resolution).

Phase 2 — Per-Channel Quantization. Rather than applying post-training quantization (PTQ) directly, we inject per-channel scale factors into the forward pass during fine-tuning [12] for a further 200 epochs. This reshapes weight distributions to align with INT8 quantization boundaries, effectively mapping continuous activation ranges into discrete integer bins before Register-Transfer Level (RTL) synthesis.

Phase 3 — Hardware Export. Scale factors are decomposed into an integer multiplier M_{int} and arithmetic right-shift S, exported as `.mem` files for direct BRAM initialization. The re-quantization operation is fused into the datapath as Eq. (1), requiring no floating-point logic at inference time. Consequently, scaling the normalized INT8 activation ensures that algorithmic operations occurring outside the neural datapath—specifically DFL decoding and NMS spatial filtering—remain numerically stable, confirming that the three-phase approach preserves detection quality constraints. Together, these four components form a complete, CPU-independent Detection Head pipeline validated in the following section.

3 Experimental Results

We independently validate the proposed architecture through cycle-accurate RTL simulation and physical synthesis. Hardware implementation targets the Zynq-7020 (xc7z020clg400-1 SoC) to evaluate logic scaling under restricted resource conditions. Design compilation is performed using Vivado 2024.2, functionally verified against a Python golden model running YOLOv8n (320×320) tested on the KIIT-MiTA UAV dataset [2].

3.1 Implementation Results

Table 1 details the PCQ-FT verification metrics. The INT8 quantization establishes a fixed-point model achieving an mAP_{50} of 67.0%. Precision increases from 77.7% to 85.3%, while recall decreases from 74.5% to 55.3%, reflecting the inherent INT8 quantization trade-off between false-positive suppression and

target recall. The final INT8 step introduces an 11.5-point mAP_{50} drop relative to the FP32 intermediate. This drop is primarily attributed to cumulative fixed-point rounding errors in the re-quantization path, an inherent constraint of INT8-only inference on edge hardware. The resulting 67.0% mAP_{50} remains sufficient for primary target bounding-box localization on the KIIT-MiTA dataset. Cycle-accurate RTL simulation against a Python golden model confirms that no additional accuracy loss occurs during hardware execution beyond the quantization penalty reported above.

Table 1. INT8 Per-Channel Quantization Fine-Tuning (PCQ-FT) Validation (KIIT-MiTA Dataset, 320×320).

Phase Calibration	**Precision**	**Recall**	**mAP_{50}**	**$mAP_{50\text{-}95}$**
YOLOv8 SiLU Baseline (FP32 640×640)	85.1%	82.6%	87.4%	58.3%
YOLOv8 SiLU Baseline (FP32 320×320)	82.2%	79.9%	84.9%	54.5%
ReLU Teacher + Distill (FP32)	77.7%	74.5%	82.8%	50.6%
PCQ-FT (FP32 Intermediate)	80.4%	68.1%	78.5%	47.9%
PCQ INT8 (RTL Target)	**85.3%**	**55.3%**	**67.0%**	**40.3%**

Table 2 documents the hardware footprint following placement and routing. The design satisfies timing closure at 105 MHz on the xc7z020 fabric, utilizing 24.1K LUTs (45.3% of available resources) and exactly 10 DSP blocks (4.55% capacity). Of the 10 DSP slices, 6 serve the CNN MAC array, 4 implement the division-free NMS multipliers, and the DFL decoder consumes none—mapping entirely to 539 LUTs via BRAM-based lookup.

Table 2. Post-Routed Device Utilization on Zynq-7020 (xc7z020clg400-1).

Programmable Logic Resource	**Slices Used**	**Available**	**Util. %**
Look-Up Tables (LUT)	24,158	53,200	45.41%
Flip-Flop Logic (FF)	25,748	106,400	24.20%
Block RAM Tiles (BRAM)	125	140	89.29%
DSP48E1 Multipliers	**10**	**220**	**4.55%**
PL Dynamic Target Power	**0.302 W**		

3.2 Throughput and Power Efficiency

System timing meets constraints at 105 MHz (Worst Negative Slack: +0.074 ns), facilitated by the five-stage load pipeline of the NMS unit. Table 3 details the cycle-accurate frame latency. Time-multiplexing the CNN components trades throughput for resource reduction, occupying 29.28 ms per frame. The post-processing hardware eliminates the CPU offload penalty: the hardware DFL decoder decodes all anchor logits in 1.56 ms, while NMS evaluates candidates in 0.28 ms. End-to-end inference for a 40×40 grid completes entirely within the PL in 31.12 ms, yielding a throughput of 32.1 FPS. Consuming 0.302 W of dynamic power, the architecture achieves 106.3 FPS/W power efficiency.

Table 3. Cycle-Accurate Latency per Detection Frame (105 MHz Constraint).

Architectural Stage	Clock Cycles	Execution Time	Throughput Share
CNN Engine (Multiplexed)	3,074,400	29.28 ms	94.1%
Hardware DFL	163,800	1.56 ms	5.0%
Hardware NMS (Division-Free)	29,400	0.28 ms	0.9%
Total Detection Head (PL)	**3,268,484**	**31.12 ms**	**100%**

3.3 Comparison with Prior Work

Table 4 compares the proposed architecture against existing hardware YOLO accelerators. Software post-processing delegation introduces significant data-transfer latency. For instance, Danilowicz et al. [9] demonstrated that an FPGA backbone sustaining 195 FPS experiences a throughput reduction to 24 FPS when restricted by CPU-bound anchor-free NMS synchronization. Direct throughput comparison across these works is precluded by differences in detection scope (head-only vs. end-to-end), model architecture (anchor-based vs. anchor-free), and FPGA platform tier. Table 4 therefore focuses on post-processing implementation strategy and DSP resource cost—the two axes most relevant to our contributions.

Table 4. Architectural Benchmark of Hardware Post-Processing Implementations.

Work	Model	Platform	HW DFL	HW NMS	Div-Free	DSP	Post-Proc
Saidani [6]	YOLOv5	Zynq-7020†	N/A	✗	N/A	—	CPU (full)
Guan [8]	YOLOv8	Zynq (HLS)†	✗	✗	N/A	—	CPU (full)
Essel [7]	YOLOv8	SoC†	✗	✗	N/A	—	CPU (full)
Danilowicz [9]	YOLOv8	ZU+ MPSoC††	✗	✗	N/A	—	CPU (full)
Anupreetham [10]	SSD	Stratix 10†††	N/A	✓	✗	>5,000	FPGA only
Nguyen [11]	NMS	Zynq-7020†	N/A	✓	✗	—	FPGA only
Ours	**YOLOv8**	**Zynq-7020†**	✓	✓	✓	**10**	**FPGA (full)**

†Edge-class (≤220 DSP) ††Mid-range SoC
†††Server-class (>5,000 DSP) — indicates metric not reported

Our implementation mitigates the CPU penalty by encapsulating NMS and DFL strictly within programmable logic, maintaining a sustained 32.1 FPS hardware pipeline. The 1.84 ms total post-processing time represents only 5.9% of frame latency, confirming that the proposed hardware integration successfully eliminates the bottleneck observed in CPU-offloaded designs. Operating at 0.302 W PL dynamic power, the architecture achieves 106.3 FPS/W—a metric unavailable to CPU-offloaded designs due to their variable software execution overhead. Furthermore, while dedicated hardware NMS accelerators [10, 11] frequently require extensive DSP allocations for IoU division networks, the proposed 10-DSP core footprint permits full Detection Head integration on the Zynq-7020 without a secondary coprocessor, directly satisfying UAV Size, Weight, and Power (SWaP) constraints. Fig. 5 illustrates qualitative detection results, confirming all five ground-truth targets are matched at an average IoU of 0.822.

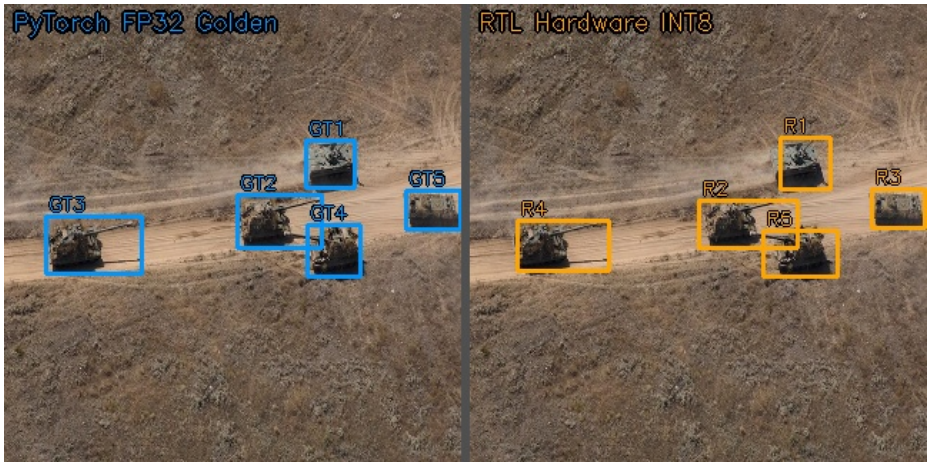

Fig. 5. Real-Time Evaluation (KIIT-MiTA 320×320). The Division-Free NMS with coordinate averaging matches all ground-truth targets at an average IoU of 0.822.

4 Conclusion

This paper proposes a hardware architecture for the YOLOv8 Detection Head, specifically optimizing anchor-free post-processing for edge FPGA integration. By eliminating hardware division from both DFL decoding and NMS, and time-multiplexing a single MAC engine across all six convolutional layers, the architecture reduces total DSP demand to just 10 blocks while executing the complete post-processing pipeline entirely within programmable logic. Cycle-accurate validation on the Zynq-7020 SoC demonstrates a sustained throughput of 32.1 FPS at a dynamic power consumption of 0.302 W, achieving 67.0% mAP_{50} on the KIIT-MiTA dataset. The hardware integration of both NMS and DFL yields a fixed 1.84 ms post-processing execution time, offering an alternative to variable-latency CPU-offloaded detection pipelines. However, BRAM saturation currently peaks at 89.3% (125/140 tiles), which presents a direct constraint on scalability for higher-resolution grids. Future work will investigate structured pruning and multi-scale head extension. Source RTL code is accessible at: https://github.com/nguyenchuyenhoa14-hub/YOLO_Tank.

References

1. Ma, C., Fu, Y., Wang, D., Fang, J.: YOLO-UAV: Object detection method of unmanned aerial vehicle imagery based on efficient multi-scale feature fusion. IEEE Access (2023)
2. Chakrabarty, S., Chatterjee, R., Chakraborty, S., Shuvo, S.R., Chowdhury, R.: Drones in defense: Real-time vision-based military target surveillance and tracking. In: Proc. 3rd Int. Conf. Intelligent Syst., Advanced Comput. Commun. (ISACC). pp. 508–513 (2025)

3. Wang, Z., Dang, C., Wang, L.: Personnel search and rescue detector based on reconfigurable FPGA accelerator: A lightweight multi-scale parallel drone-mounted detector. IEEE Geoscience Remote Sensing Letters 22 (2025)
4. Ultralytics: YOLOv8 by Ultralytics. https://github.com/ultralytics/ultralytics (2023)
5. Li, X., Wang, W., Wu, L., Chen, S., Hu, X., Li, J., Tang, J., Yang, J.: Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 33, pp. 21002–21012 (2020)
6. Saidani, T., Ghodhbani, R., Alhomoud, A., Alshammari, A.: Hardware acceleration for object detection using YOLOv5 deep learning algorithm on Xilinx Zynq FPGA platform. Eng., Technol. & Appl. Sci. Res. 14(1), 13066–13071 (2024)
7. Essel, E., Lacy, F., Belu, R., Areed, N.F.F., Ismail, Y.: Performance optimization of FPGA-accelerated YOLOv8 for driver drowsiness detection using Vivado HLS. Int. J. Comput. Digital Syst. (IJCDS) (2025)
8. Guan, Z., Xie, M., Liu, B., Yang, Z.: YOLOv8 detection head improvements for FPGA deployments. In: 2024 IEEE 17th International Conference on Signal Processing (ICSP). pp. 1–5. IEEE (2024)
9. Danilowicz, M., Kryjak, T.: Real-time multi-object tracking using YOLOv8 and SORT on a SoC FPGA. arXiv preprint arXiv:2503.13023 (2025)
10. Anupreetham, A., et al.: End-to-end FPGA-based object detection using pipelined CNN and non-maximum suppression. IEEE Access (2024)
11. Nguyen, D.A., Tran, T.H., Hoang, T., Nguyen, X.T.: A fast and resource-friendly hardware accelerator for non-maximal suppression. In: 2021 IEEE International Symposium on Circuits and Systems (ISCAS). pp. 1–5. IEEE (2021)
12. Jacob, B., Klinker, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR) pp. 2704–2713 (2018)